从这里开始

DASE7503 · 导航 / 视觉 / EC · 现场操作版 · 2026-11-02

本次运行方式

真机只用前进、后退和转向，横向速度 vy=0。先验证一个货架，再扩大到四个。所有命令均在 Ubuntu 24.04 / ROS 2 Jazzy 终端中执行。

你现在从哪一步开始？

现场情况	从这里开始
还没装代码，或设备型号不清楚	第 3 页登记设备 → 第 4–5 页准备环境
软件已准备好，首次连接 EC	第 6 页通信 → 第 7 页轮测 → 第 8 页升降
EC 已通过，开始接传感器	第 9 页雷达 → 第 10 页相机 → 第 11–12 页实测参数
全部输入齐全，准备动车	第 13–14 页空载 → 第 15–16 页单货架
单货架已通过	第 17–18 页四货架及避碰 → 第 19–21 页故障、记录和验收

可以点击跳转

终端怎么分工 / 怎么停止 → 第 2 页

更新代码与安装 → 第 4 页

第一次连接 EC → 第 6 页

单货架启动 → 第 16 页

故障恢复 → 第 19 页

验收勾选表 → 第 21 页

阅读方法

每页只完成一个阶段。蓝色标题告诉你在哪个终端操作；灰底框里的命令逐块执行。“保持运行”的终端不要关；观察命令看完后按 Ctrl+C，再执行下一块。

尚未知晓的雷达、相机型号和 ROS 主机留在表格中填写。PDF
填写内容不会自动写进配置文件。前一阶段未通过，就停在该阶段处理。软件和历史仿真不代替现场验收。

终端分工与停止方法

常用页 · 每个终端只承担固定用途

终端	固定用途	什么时候保持运行
A · 通信	EC 串口桥 + 适配器	连接 EC 后持续运行；切换运动/升降权限时重启
B · 主程序	模型显示、独立视觉、空载 NAV、完整任务	每次只启动其中一个；换阶段先 Ctrl+C
C · 操作	查状态、发一次指令、使能、停车	命令结束后可输入下一条
D · 雷达	实际雷达驱动	传感器阶段及完整任务持续运行
E · 相机（可选）	外部相机驱动	只有不用项目自带相机启动时才开
F · 观察/记录（可选）	持续显示反馈或录包	需要边发指令边看结果时另开

每开一个新终端，都先执行（第 5 页创建此文件）

```
source ~/dase7503-commissioning/session.env
```

出现异常：先使用现场硬件急停，确认实物已停

下面两条是不同的软件请求，可在终端 C
执行。服务有回复也必须观察实际车轮和升降机构，不能把回复文字当作物理停止确认。

```
ros2 service call /nav/emergency_stop std_srvs/srv/Trigger '{}'
```

NAV 运行时可用；停车会锁存，排除原因后重启 NAV 才能恢复。

```
ros2 service call /ec_bridge/estop std_srvs/srv/SetBool \
  '{data: true}'
```

正常收工

先确认机构稳定、车辆静止，在 C 解除使能；再结束 B、A 及驱动进程。Ctrl+C
只表示退出软件，不能替代升降保持或硬件急停。

```
ros2 service call /ec_bridge/set_arm std_srvs/srv/SetBool \
  '{data: false}'
```

填写设备信息

准备 1 / 3 · 不知道的先留空，到对应阶段必须补齐

操作人 / 日期

直接连接 EC 的 ROS 2 主机 / 操作系统

EC 板卡 / 实际固件 commit

串口完整路径（优先 /dev/serial/by-id/...）

实际波特率（代码默认 115200）

雷达型号 / 接口

雷达驱动与启动命令

相机型号 / USB 或排线

相机驱动与启动命令

硬件急停 / 断电操作位置

开始前

按 EC 实际接线表核对电源、电机、编码器、IMU、升降及急停。GPIO UART 需核对引脚、共地及逻辑电平；不能套用 USB 接线。初次动作由一人操作、一人观察并看守急停。

☐ 接线、固定、急停位置及四轮离地支撑已检查

Windows / WSL 只能使用已透传到 Ubuntu 中的设备；Windows 的 COM 名称不是 Linux 串口路径。未知型号不在这里猜驱动命令。

更新代码并编译

准备 2 / 3 · 终端 C · 只需在安装或更新代码时执行

1 进入仓库（已有仓库只执行这一块）

```
cd ~/dase7503-group-project
```

如果仓库实际在其他目录，改为那个目录。没有仓库时，先执行下面两条，然后继续第 2 步。

```
git clone https://github.com/zwkcyy/dase7503-group-project.git
cd dase7503-group-project
```

2 更新 main

```
git status --short
git fetch origin
git switch main
git pull --ff-only
```

有本地修改时

git status 有输出，或 switch / pull
报错时，先保存并处理现有修改；不要强制覆盖。真机联调使用合并后的 main。

3 安装依赖 → 编译 → 下载扫码模型

```
bash scripts/nav.sh install
```

等安装结束、没有错误后，再执行下一块。需要 ROS 2 Jazzy 已安装。

```
bash scripts/nav.sh build
```

```
bash ros2_ws/src/vi/robot_vision/scripts/download_qr_models.sh
```

通过条件

安装和下载成功；编译无失败项，包含 ec_serial_bridge。默认安装目录是 ~/dase7503-nav-work/install。普通安装/编译命令不自动执行联调或测试。

☐ 代码已更新、依赖已安装、编译及扫码模型下载完成

设置每个终端的环境

准备 3 / 3 · 终端 C · 复制一次，后面只需加载

1 创建配置目录并打开文件

```
mkdir -p ~/dase7503-commissioning
nano ~/dase7503-commissioning/session.env
```

2 把下面内容粘贴到文件中，并修改实际路径与波特率

```
source /opt/ros/jazzy/setup.bash
source ~/dase7503-nav-work/install/setup.bash
export ROS_DOMAIN_ID=76
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
export REPO="$HOME/dase7503-group-project"
export CFG="$HOME/dase7503-commissioning"
export EC_PORT="/dev/ttyACM0"
export EC_BAUD=115200
export CAMERA_MANAGED=true
export MAP_FILE=""
export X0=""
export Y0=""
export YAW0=""
```

REPO 改成实际仓库路径，EC_PORT 改成实际串口；EC_BAUD 按 EC 固件填写。X0 / Y0 / YAW0 先留空，到第 12 页填实测起点。

CAMERA_MANAGED=true：项目启动 camera_ros。已有其他相机驱动时设 false，并在 E 启动它。MAP_FILE 留空使用课程墙体地图；使用实测地图则填地图 YAML 的绝对路径。

3 保存并加载

nano：Ctrl+O → Enter 保存，Ctrl+X 退出。每个新终端都执行：

```
source ~/dase7503-commissioning/session.env
```

通过条件

终端中 echo "$ROS_DISTRO" 输出 jazzy，REPO 和串口路径正确；所有终端使用同一份 session.env。修改文件后，已开的终端也重新加载。

本册默认所有进程在同一 ROS 主机；可用多个 SSH 窗口。跨主机需另行配置发现范围、网络及时间同步，不能直接沿用 LOCALHOST。域 76 与旧仿真域 75 分开。

先接通 EC，不允许动作

阶段 1 · A 保持通信；C 观察 · 尚未启动导航

1 C：找串口，填好第 5 页的 EC_PORT

```
ls -l /dev/serial/by-id/
```

没有 by-id 时检查 /dev/ttyACM* 或 /dev/ttyUSB*。权限不足：sudo usermod -aG dialout "$USER"，然后重新登录。确保只有一个程序打开该串口。

2 A：启动并保持运行

```
cd "$REPO"
bash scripts/nav.sh ec start_bridge:=true \
  port:="$EC_PORT" baudrate:="$EC_BAUD" \
  enable_motion:=false enable_lift:=false
```

3 C：读取状态（每条收到一条后自动结束）

```
ros2 topic echo /ec_bridge/status --once
ros2 topic echo /ec_adapter/status --once
ros2 topic echo /ec/lift_state --once
ros2 topic echo /odom/wheel --once
ros2 topic echo /imu/data_raw --once
```

看到这些才算通过	没有看到时查什么
open=true，link_ok=true；接收计数增加	供电、串口、权限、波特率、固件协议
ec_fault=0；状态不是 null	null 是缺失或过期的 84 帧，不是正常值
telemetry_ready=true	查看 issues；检查时间戳、帧名、里程计和 IMU
lift_completion_source = ec_timed_state_transition	若不是，检查是否加载了旧配置或旧安装目录
allow_lateral_motion=false	桥、适配器都应禁用横移；关闭旧节点

命令一直等待表示尚未收到消息，可 Ctrl+C 返回处理。坐标：x 前、y 左、z 上；正 yaw 为俯视逆时针。ROS 使用 m、rad、m/s、rad/s。

- [ ] 通信、里程计、IMU 和真实升降状态均收到且有效

四轮离地，检查运动方向

阶段 2 · 只允许架空轮测 · 升降禁用

动作前

可靠支撑整车，使四个轮子全部离地。bench_motion
绕过地面导航避障，只能用于本页架空轮测；不能用它驾驶落地车辆。

1 A：Ctrl+C 结束旧通信，再重启

```
cd "$REPO"
bash scripts/nav.sh ec start_bridge:=true \
  port:="$EC_PORT" baudrate:="$EC_BAUD" \
  enable_motion:=true enable_lift:=false
```

2 C：使能，并确认状态已实际改变

```
ros2 service call /ec_bridge/set_arm std_srvs/srv/SetBool \
  '{data: true}'
ros2 topic echo /ec_bridge/status --once
```

必须看到 ec_armed=true、ec_estop=false、ec_fault=0。只看到 frame sent 不算使能成功；若立即掉回未使能，先查固件看门狗。

3 C：先只发一次前进脉冲，观察一秒后停止

```
ros2 run robot_bringup bench_motion \
  --wheels-raised --forward 0.04 --seconds 1
```

通过后，逐次把 --forward 0.04 改为 --forward -0.04、--turn 0.15、--turn -0.15。每次都等停止再执行下一次，不把四次动作连着粘贴。

通过条件

四轮组合分别对应前进、后退、左转、右转；轮式里程计 vx / wz 符号正确。工具会拒绝其他 /cmd_vel 发布者以及过期 EC 状态。车身未转时 IMU 接近零是正常的。

按第 2 页解除使能后，人工推动车身比较实测与里程计距离；缓慢逆时针转车时 IMU gz 应为正。轴向错误需修正 EC / IMU 配置。

实测距离 / 里程计距离 / 四轮组合与 IMU 方向

☐ 四轮脉冲后能停止，方向和里程计检查通过

空载升降，核对实际反馈

阶段 3 · 运动禁用 · 升降空载且行程内无阻挡

1 A：结束旧进程，按本页权限重启

```
cd "$REPO"
bash scripts/nav.sh ec start_bridge:=true \
  port:="$EC_PORT" baudrate:="$EC_BAUD" \
  enable_motion:=false enable_lift:=true
```

C 按第 7 页使能并确认状态。/ec/lift_state 应与实物对应：底端 0、顶端 1。初始在顶端则先下降。

2 F：持续看任务侧反馈，保持运行

```
ros2 topic echo /lift_state
```

3 C：实际在底端时，只发一次上升

```
ros2 topic pub --once /lift_cmd std_msgs/msg/Int8 '{data: 1}'
```

上升通过

F 先显示 -1；本次 F0 回执和 EC 从 0 到 1 的新变化齐全后，才显示 1。观察机构确已到顶停止；主机不会仅等三秒就假报成功。

4 C：实物停止且反馈 1 后，只发一次下降

```
ros2 topic pub --once /lift_cmd std_msgs/msg/Int8 '{data: 0}'
```

下降应显示 -1 → 0，且机构实际回底端。同端点重复同向指令会拒绝；不得连续发布或用人工 /lift_state 跳过反馈。

5 C：在空载条件下核对停止能力

```
ros2 service call /ec_adapter/stop_lift std_srvs/srv/Trigger '{}'
```

预期 success=false：固件已取消 07；代码只取消未来重发，升降可能继续。04、05 及急停效果先空载实测，不以载货断电验证保持力。

上升 / 下降实测时间与反馈

04 / 05 / 硬件急停的空载实际效果

☐ 回执、EC 状态和实物一致；停止能力已记录

接雷达，检查方向和盲区

阶段 4 · D 运行雷达；B 显示模型；C 观察

1 D：填写并执行第 3 页真实雷达驱动命令

按实际型号安装官方 ROS 2 Jazzy 驱动；输出 sensor_msgs/msg/LaserScan 到 /scan。话题不同则在驱动端 remap。型号和命令未知时，停在本步补齐。

2 C：先看类型和一帧数据，再观察频率

```
ros2 topic info /scan --verbose
ros2 topic echo /scan --once
```

```
ros2 topic hz /scan
```

频率看稳定后 Ctrl+C。核对时间戳持续更新、有效测距、角度顺序和测距范围。

3 B：启动独立模型显示，保持运行

```
ros2 launch robot_bringup cad_view.launch.py
```

4 C：检查模型到雷达测量中心的变换

```
ros2 run tf2_ros tf2_echo base_link laser
```

默认参考帧 laser；实际 /scan.frame_id 不同则按实际帧检查。物体依次放在车前、左、右，RViz 点云方向应一致。CAD 原点不一定是测量中心，实测修正第 11 页几何文件。

通过条件

方向和 TF 正确，空载/载货时外部货架腿可见，后方退出区域有有效观测。后续 NAV 中继续核对过滤后的 /scan_filtered 没误删障碍。central_rear_scan_unknown 不可通过关保护绕过。

雷达测量中心 xyz(mm) / rpy(rad)

空载 / 载货 / 后方遮挡记录

☐ 雷达方向、测量中心与盲区已检查

本页结束在 B 按 Ctrl+C；启动 NAV 前关闭独立模型显示以避免重复 TF。只由 EKF 发布 odom → base_link，AMCL 发布 map → odom。

接相机，确认六张二维码

阶段 5 · B 独立视觉；C 触发；F 看结果

1 准备图像输入

CAMERA_MANAGED=true 时尝试项目 camera_ros；是否支持取决于实际相机。若使用外部驱动，设 false，在 E 启动已确认的驱动，输出 /camera/image_raw。不能出图就先按型号处理驱动。

2 B：启动视觉；需要弹窗且有桌面时用 true

```
ros2 launch robot_vision vision_bringup.launch.py \
  start_camera:="$CAMERA_MANAGED" \
  image_topic:=/camera/image_raw show_results:=true
```

无桌面 / SSH 环境把 show_results 改为 false。C 可用 ros2 topic hz /camera/image_raw 看图像频率，观察后 Ctrl+C。

3 F：先等待结果；C：面对一张码，再触发扫描

```
ros2 topic echo /qr/data
```

```
ros2 service call /qr/start_scan std_srvs/srv/Trigger '{}'
```

上一块在 F，下一块在 C。依次扫描 START、实际 A/B/C/D
货架码、END；每换一张都重新触发。窗口保留旧内容只是显示行为，不代表又识别了一次。

通过条件

结果与实物完整一致；货架格式 RACKA_后缀 至
RACKD_后缀，后缀为字母数字且大小写敏感。/qr/data
只有一个检测器发布者；现场距离、角度、光照下可重复识别。

A 的完整二维码

B 的完整二维码

C 的完整二维码

D 的完整二维码

可靠距离 / 视角 / 现场光照

结束：C 调用 /qr/stop_scan（同 start_scan 的 Trigger 格式），B 按 Ctrl+C
关闭独立视觉。完整任务会自行启动检测器并控制扫描；外部相机 E 可以保留。

填写真实车身与传感器尺寸

阶段 6 / 2 页 · 终端 C · 保存现场副本

1 复制几何模板（保留仓库基准）

```
 cp -n "$REPO/ros2_ws/src/nav/robot_description/config/"\
"mechanical_cad_20261025.yaml" "$CFG/geometry.yaml"
 nano "$CFG/geometry.yaml"
```

检查项	填写要求
空载外廓	包含最外侧部件和前置雷达突出部分
loaded_bounds_mm	包含整车和搬起后的整个货架；不能为 null
雷达 / 相机 / IMU	填测量中心或光心相对 base_link 的位置、姿态
confirmed	只有实际核对后才设 true

尺寸与坐标

几何文件标为 mm 的项目用毫米；ROS 位姿和任务坐标用米、弧度。base_link 是模型参考原点，传感器测量中心未必就是零件建模原点。

空载 [xmin, ymin, xmax, ymax] mm

载货 [xmin, ymin, xmax, ymax] mm

相机光心 xyz(mm) / rpy(rad)

IMU 测量中心 xyz(mm) / rpy(rad)

车身高度 / 货架底部净空 / 载货旋转余量

通过条件

geometry.yaml 包含已测量的完整外廓与安装数据；实车与模型的前后左右一致。后续空载/载货时，两张代价地图的 footprint 都须覆盖真实最外点。

原 CAD 有高度与底部净空限制，不能用缩放模型隐藏。模型可作为开发基准，最终需量实物；空载初步定位可用 empty_only，完整搬运必须有载货数据。

填写货架位置与实际起点

阶段 6 / 2 页 · 终端 C · 未填完不得启动任务

1 复制任务模板并填写

```
 cp -n "$REPO/ros2_ws/src/common/robot_bringup/config/"\
"mission_hardware_template.yaml" "$CFG/mission_all.yaml"
 nano "$CFG/mission_all.yaml"
```

参数	填写要求
rack_qr_codes	第 10 页四张完整实际码，不使用 DEMO
rack_x/y_m、rack_yaw_rad	A～D 对准时 base_link 的实际位姿（非架子中心）
placement_x/y_m、placement_yaw_rad	四个不同放置位；互不重叠，后续路径可通过
docking_distance_m / placement_exit_distance_m	0.42 m 是初始默认；实测退出到能安全旋转。当前退出累计上限 0.50 m
retreat_after_pickup	保持 true；检查取架和放架后的退出空间
coordinates_confirmed	所有 null、空码、占位值填完并检查后设 true

2 填起点到 session.env，再给每个终端重新加载

填 export X0=实际米数、export Y0=实际米数、export YAW0=实际弧度。等号右侧必须是测得的数字；本册不预填会让车走错位置的示例坐标。

```
nano "$CFG/session.env"
```

```
source ~/dase7503-commissioning/session.env
```

实际起点 x(m) / y(m) / yaw(rad)

地图要一致

课程地图原点在内侧左下角；x 沿 3020 mm 边、y 沿 2000 mm 边。实测地图可在 MAP_FILE 指定。默认地图是墙体；生成器的货架矩形只画预览，其他货架和已放下货架需激光代价地图观测。

☐ 四架码、取放位、退出余量、实际起点与坐标系已核对

先启动空载导航

阶段 7 / 2 页 · 车落地前确认前面阶段全部通过

1 清理旧进程，保持真实雷达运行

关闭 B 中的独立模型显示和独立视觉。D
雷达继续。车空载，场地受控，硬件急停可立即操作。不要同时启动 hardware
包装入口，以免重复适配器或 TF。

2 A：结束旧进程，允许运动、禁止升降

```
cd "$REPO"
bash scripts/nav.sh ec start_bridge:=true \
  port:="$EC_PORT" baudrate:="$EC_BAUD" \
  enable_motion:=true enable_lift:=false
```

3 B：核对起点已填写，再启动唯一一份 NAV

```
: "${X0:?Fill X0}" "${Y0:?Fill Y0}" "${YAW0:?Fill YAW0}"
EKF="$(ros2 pkg prefix robot_bringup)/share/robot_bringup"
ros2 launch robot_navigation bringup.launch.py \
  geometry_file:="$CFG/geometry.yaml" empty_only:=true \
  initial_loaded:=false drive_mode:=differential rviz:=true \
  initial_x:="$X0" initial_y:="$Y0" initial_yaw:="$YAW0" \
  map_file:="$MAP_FILE" \
  ekf_params_file:="$EKF/config/ekf_ec_10hz.yaml"
```

此时先不要点目标

等待 NAV 和定位启动。RViz
中核对墙体、激光和车身方向；下一页先做状态检查，通过后才使能并选择近距离目标。

B 保持运行，去下一页的 C 操作

Fill X0 / Y0 / YAW0 表示变量未填写或本终端还没重新
source。其他启动报错先保留日志、查看缺项；不要补仿真节点或假数据来让真机检查变绿。

空载导航通过后才搬运

阶段 7 / 2 页 · C 检查；B 的 NAV 保持运行

1 C：核对运动模式和软件通路

```
ros2 param get /controller_server FollowPath.motion_model
ros2 param get /amcl robot_model_type
ros2 param get /velocity_smoother max_velocity
cd "$REPO"
bash scripts/nav.sh robot-doctor --mode navigation \
  --empty-only --geometry-file "$CFG/geometry.yaml"
```

预期：DiffDrive、DifferentialMotionModel、[0.15, 0.0, 0.4]。doctor 通过只表示软件输入齐全。

2 C：逐块观察，结束每一块用 Ctrl+C

```
ros2 run tf2_ros tf2_echo map base_link
```

```
ros2 topic hz /scan_filtered
```

通过条件

定位不跳变；激光与墙体重合；车身朝向正确；空载 footprint 包住实车；/scan_filtered 保留外部障碍。只有 EKF 发布 odom → base_link。

3 C 使能并确认，然后在 RViz 点近距离前方目标

按第 7 页 set_arm 命令使能，确认 ec_armed=true、ec_estop=false、ec_fault=0。先短直线，再小角度转向；不从背后或障碍旁目标开始。

```
ros2 topic echo /cmd_vel
```

linear.y 应一直为 0。看完
Ctrl+C；动作相反、定位跳变、墙体不符时立即停。普通空载阶段的倒车可能因未授权而被保护拒绝。

4 C：演练软件停车；排除原因后重启 NAV

```
ros2 service call /nav/emergency_stop std_srvs/srv/Trigger '{}'
```

☐ 定位、短直线、小角度转向、零横移与软件停车通过

先准备 A 单货架任务

阶段 8 / 2 页 · C 配置；A 保持唯一一份通信

1 从已经填好的四货架文件生成单货架副本

```
python3 - <<'PY'
import os, pathlib, yaml
d = pathlib.Path(os.environ["CFG"])
data = yaml.safe_load((d/"mission_all.yaml").read_text())
p = data["delivery_mission"]["ros__parameters"]
keys = ("rack_labels", "rack_qr_codes", "rack_x_m", "rack_y_m",
        "rack_yaw_rad", "placement_x_m", "placement_y_m",
        "placement_yaw_rad")
for key in keys:
    p[key] = p[key][:1]
(d/"mission_one.yaml").write_text(
    yaml.safe_dump(data, sort_keys=False), encoding="utf-8")
PY
```

2 关闭 B 的独立 NAV / 视觉，按实际起点摆放车辆

车空载、升降底端；四个真实货架均放在实测位置。确认 A 入口、目标放置位和退出空间。完整系统会自己启动 NAV 和视觉，不能与旧进程同时运行。

3 A：结束旧进程，同时允许运动和升降

```
cd "$REPO"
bash scripts/nav.sh ec start_bridge:=true \
  port:="$EC_PORT" baudrate:="$EC_BAUD" \
  enable_motion:=true enable_lift:=true
```

A 对准位姿 x(m) / y(m) / yaw(rad)

A 放置位姿 x(m) / y(m) / yaw(rad)

取架 / 放架退出距离及旋转外廓余量

确认后再去下一页

D 雷达保持运行；如用外部相机，E 保持运行。不得人工发布 /lift_state 假装举升完成；必须由真实反馈更新载货状态及两张代价地图。

运行 A 的完整搬运循环

阶段 8 / 2 页 · B 启动；C 操作；F 看任务状态

1 B：先启动系统，暂不展示 START

```
cd "$REPO"
: "${X0:?Fill X0}" "${Y0:?Fill Y0}" "${YAW0:?Fill YAW0}"
bash scripts/nav.sh robot \
  geometry_file:="$CFG/geometry.yaml" \
  mission_file:="$CFG/mission_one.yaml" \
  drive_mode:=differential initial_loaded:=false \
  initial_x:="$X0" initial_y:="$Y0" initial_yaw:="$YAW0" \
  map_file:="$MAP_FILE" \
  start_camera:="$CAMERA_MANAGED" show_results:=true rviz:=true
```

无桌面时把 show_results 和 rviz 都改 false，并用已配置好的可视化观察方式。EC 通信 A 在此命令之外独立运行。

2 F：等待任务状态；C：确认输入、使能和实际状态

```
ros2 topic echo /mission/status
```

系统检查通过、任务进入 WAIT_START 后，C 按第 7 页使能并核对 ec_armed / ec_estop / ec_fault。确认轮廓、图像与激光正常，才展示 START。

3 观察完整顺序

取架	放架
到 A 接近位 → 核对 A 完整码	到放置接近位 → 直线入位
直线入架 → 举升 → 真实反馈	核对 END → 放下 → 真实反馈
两张地图变为载货 footprint → 退出	两张地图变为空载 footprint → 退出

通过条件

A 完成一次完整循环，任务 COMPLETE；无擦碰、拖架，实物位置和状态一致。车辆在两条架腿之间不原地掉头。

alignment refused：任务会停车报错；检查接近位、定位、轮滑和实际对准误差，在架外重新对准。不要加大容差掩盖机械间隙不足。

- [ ] A 单货架完整循环通过

再运行四个货架

阶段 9 / 2 页 · 单货架全部通过后执行

1 停止 B，确认机构稳定，重新布置实物

车辆回到填写的实际起点；A/B/C/D 放回规定位置，四个放置位清空。若起点改变，修改 session.env 并给各终端重新加载；确认升降在底端、车为空载。

2 B：同样启动，只换成 mission_all.yaml

```
cd "$REPO"
: "${X0:?Fill X0}" "${Y0:?Fill Y0}" "${YAW0:?Fill YAW0}"
bash scripts/nav.sh robot \
  geometry_file:="$CFG/geometry.yaml" \
  mission_file:="$CFG/mission_all.yaml" \
  drive_mode:=differential initial_loaded:=false \
  initial_x:="$X0" initial_y:="$Y0" initial_yaw:="$YAW0" \
  map_file:="$MAP_FILE" \
  start_camera:="$CAMERA_MANAGED" show_results:=true rviz:=true
```

3 核对状态和使能后展示 START；观察已放架的影响

F 继续显示 /mission/status。从 A 放下开始，它就成为后续路线的障碍；B、C、D 的路线和转弯必须避开它以及其他货架。需要时另开已加载环境的终端观察 /nav/motion_safety。

通过条件

四个不同货架各完成一次，四个放置位不重叠；最终 COMPLETE；无碰撞、擦碰、拖架或人工介入。仍须完成下一页专项观察和第 19 页故障恢复。

任务默认 180 s。当前低速联调配置不保证达到时间目标；功能和空间检查通过后，才根据真实停车距离与路线记录调整速度。超时 ERROR 不能记作通过。

A / B / C / D 结果及异常

四货架耗时 / 擦碰次数 / 人工介入次数

☐ 四货架循环通过，结果与实物一致

把货架、载货和倒车一起检查

阶段 9 / 2 页 · 运行时观察，必要时单独低速复核

检查场景	现场怎样判断
其余三货架	global / local costmap 能看到架腿或边界，规划与整个车身外廓不相交
已经放下的 A	保持 A 在真实放置位；B 后续接近与返回路线仍避开 A，再扩展到 C/D
载货外伸和转弯	两张地图的 footprint 覆盖最外点；转弯扫过范围也包含货架外伸
空载正向停车	从足够远处低速接近固定柔性障碍，记录实际停止距离和余量
载货正向停车	重做相同检查；记录载货制动和扫描延迟
倒车退出	只在 EXIT_RACK / EXIT_DESTINATION，低速且限距；后方中央扇区有效，整条退出区域受控

盲区的处理

空载、举升后、放下后都检查前置雷达遮挡。central_rear_scan_unknown / missing_load_state 出现时处理观测或状态来源，不关闭保护。过滤掉车体不会恢复被车体挡住的空间。

现有后方保护只覆盖有限的中央扇区，不能证明整个后方无盲区。退出区内不得有人员或新物体进入未观测位置；要消除此限制，需要补充实际后向观测。不要用人充当障碍物。

空载速度 / 停车距离 / 剩余距离

载货速度 / 停车距离 / 剩余距离

退出方向可观测范围 / 未覆盖盲区

已放下货架对后续路径与转弯的影响

☐ 空载、载货、其余货架及已放下货架均检查通过

故障发生时怎么恢复

阶段 10 · 先在架空或空载受控条件下演练

看到什么	先做什么 / 怎样恢复
串口断开	确认 EC 看门狗实际停车；重连后只请求状态，不重放旧升降、不自动使能。核对实际位置后重新使能并发新指令
F0 回执丢失或拒绝	未确认指令最多重发 5 次，总有效期 2 s；不报成功。检查 EC 回执，不增加无限重试
升降反馈中断	不算完成；实物可能还在走原行程。先稳定机构，确认位置，再清除任务并重启
错码或相机断流	扫码阶段超时应 ERROR；不能用上一次结果冒充。恢复图像、检查实际码，再重新开始
雷达 / TF / 里程计过期	motion_guard 应输出零速度并给原因；修复驱动、时间戳和 TF，不关闭保护
软件急停	/nav/emergency_stop 会锁存。按第 2 页确认实际停止，查原因后重启 NAV
任务 ERROR 或超时	不会自动放下正在搬运的货架。先稳住实物与机构，再布置场地、恢复反馈、重启任务

每次恢复前都要检查

实际静止、机构稳定、故障已排除、状态新鲜。EC 急停释放按实际固件规则操作，随后核对 ec_armed / ec_estop / ec_fault；不直接释放后继续跑旧任务。

升降停止能力有限，软件退出不保证中途停住。升降故障先空载演练，不做载货断电试验。EC 急停与看门狗的物理效果必须现场记录。

断线 / 重连 / 不重放旧指令的实际结果

反馈中断 / 错码 / 传感器过期的实际结果

急停 / 任务错误 / 恢复结果与未解决项

把过程和参数保存下来

阶段 11 · 每次正式运行前开始记录

1 F：保存版本与配置副本

```
RUN="$HOME/dase7503-records/$(date +%Y%m%d-%H%M%S)"
mkdir -p "$RUN"
git -C "$REPO" rev-parse HEAD > "$RUN/nav_commit.txt"
cp "$CFG/geometry.yaml" "$RUN/"
cp "$CFG/mission_all.yaml" "$RUN/"
cp "$CFG/session.env" "$RUN/"
```

如果实际跑单货架，还需 cp "$CFG/mission_one.yaml" "$RUN/"。实际 EC 固件 commit 填入下框并保存在运行记录中。

2 F：可选录包，保持运行至本次任务结束

```
ros2 bag record -o "$RUN/bag" \
  /mission/status /ec_bridge/status /ec_bridge/lift_event \
  /ec_adapter/status /ec/lift_state /lift_state /lift_cmd \
  /cmd_vel /odom/wheel /imu/data_raw /odometry/filtered \
  /scan /scan_filtered /tf /tf_static /qr/data /nav/loaded \
  /nav/motion_safety
```

结束按 Ctrl+C，等保存完成再关闭终端。若 F 正在持续看状态，另开一个加载环境的终端录包；相机原图较大，只在查漏码时另录。

3 找到任务自动记录，连同本册填写结果保存

完整系统的 run_recorder 通常保存到 ~/.ros/nav_vi_runs/。保存对应运行目录、实测配置、NAV/EC 版本及现场记录。PDF 中填了数据，不代表 YAML 已修改。

本次录包 / 自动记录目录

主仓库实际 commit

EC 板上实际固件 commit

本次配置版本 / 现场照片或视频位置

☐ 本次版本、实际配置与运行记录均已保存

完成后勾选验收

收尾 · 未通过的写明原因，不默认打勾

- [ ] 通信、FO、真实状态、断线重连后不重放通过
- [ ] 轮序、前后/转向、轮式里程计与 IMU 方向通过
- [ ] START、四张实际货架码、END 及错误码处理通过
- [ ] 空载定位、差速控制、入架对准和软件停车通过
- [ ] 升降实物与反馈一致；物理停止能力已记录
- [ ] 载货外廓、其他货架、已放架及倒车盲区已检查
- [ ] 单货架与四货架完整循环通过
- [ ] 断线、过期数据、反馈丢失和任务错误恢复通过
- [ ] 版本、参数、日志、录包和现场记录已保存

四货架耗时 / 擦碰次数 / 人工干预次数

尚未通过的项目与原因

下一步动作 / 负责人 / 日期

NAV / EC / 现场观察人确认

完成的含义

所有实际执行的结果、已知盲区和未通过项都有记录。前进方向正确、软件检查绿灯或任务 COMPLETE 中任何一项，都不能单独代替完整真机验收。

填写后另存 PDF，保留空白原版供下一次联调使用。可填写框和勾选框保持可编辑；中文输入效果取决于阅读器，必要时使用支持中文表单的 PDF 阅读器或打印填写。

附录：当前决定与文件入口

需要排查代码时再看 · 现场先按前面步骤操作

已经确定的项目	当前配置
运动模式	真机 differential，vy=0；AMCL DifferentialMotionModel，MPPI DiffDrive
通行速度上限	前进 0.15 m/s；转向 0.4 rad/s
直线入架 / 退出	最高前进 0.08 m/s、后退 0.06 m/s；不在架腿间掉头
对准门槛	横向误差达到 8 mm 或角度达到 0.018 rad 拒绝；仍需小于实际机械余量
意外横向命令	整条指令拒绝并输出零速，不只删掉 vy 后继续
通信与升降	二进制 UART 桥；完成依据为本次接受回执 + 新端点变化。EC 端点为计时状态，非独立到位传感器

入口 / 文件	用途
scripts/nav.sh ec / robot	分别启动 EC 通信、完整任务
ec_20261102.yaml / ec_bridge_20261102.yaml	适配器反馈与桥参数；停止能力开关默认关闭
mission_hardware_template.yaml	四货架现场填写模板
robot_geometry/bringup.py	AMCL、MPPI、速度平滑器与运动保护覆盖
robot_mission/docking_controller.py	差速直进直退及对准拒绝

证据边界

这次代码已做过 10 包编译与 Python 语法编译；新的 differential 真机模式尚未完成实车或 Gazebo 回归。GitHub 自动检查结果见对应 PR。旧 Omni 仿真不等于新模式通过。

EC 私有固件子仓库未能独立读取，其固定提交未修改。依据主仓库 PR #18 的桥及本次整合；F0、04/05、轴向、端点语义均须与实板核对。

参数依据：Nav2 Jazzy 官方文档的 MPPI motion_model 和 AMCL robot_model_type；本册命令以合并到 main 的仓库代码为准。